



Actividad 4

Nombre Alumno: Abish Abril Santana García

Materia: Estructura de datos

Nombre Profesor:

Rojas Perea, Adalberto Emmanuel



Descripción

Implementarás un árbol binario en Java, realizando operaciones básicas (**inserción, eliminación, búsqueda**), y demostrarás diferentes tipos de recorridos.

Adicionalmente, gestionarás un sistema de empleados utilizando árboles binarios para optimizar la búsqueda y gestión de información de los empleados.

Objetivo

- Comprender la estructura y funcionamiento de los árboles binarios.
- Implementar y manipular árboles binarios en Java.
- Aplicar los conceptos aprendidos en un caso práctico: gestión de empleados.

Instrucciones

1. Implementación del árbol binario:

- Crea una clase **`Nodo`** que represente los nodos del árbol.

```
class Nodo {  
    int valor;  
    Nodo izquierdo;  
    Nodo derecho;  
  
    // Constructor  
    public Nodo(int valor) {  
        this.valor = valor;  
        this.izquierdo = null;  
        this.derecho = null;  
    }  
}
```

- Crea una clase **`ArbolBinario`** que incluya métodos para insertar, eliminar y buscar nodos, así como los diferentes tipos de recorridos.

```
// Método para insertar un nodo  
public void insertar(int valor) {  
    raiz = insertarRecursivo(raiz, valor);  
}
```

```
// Método para buscar un valor  
public boolean buscar(int valor) {  
    return buscarRecursivo(raiz, valor);  
}
```

```
// Método para eliminar un nodo  
public void eliminar(int valor) {  
    raiz = eliminarRecursivo(raiz, valor);  
}
```

2. Realización de operaciones:

- Inserta elementos en el árbol y muestra el resultado.

```
// En el main:
arbol.insertar(50);
arbol.insertar(30);
arbol.insertar(70);
// Luego mostramos con los recorridos
arbol.inorden(); // Muestra: 20 30 40 50 60 70 80
```

- Elimina elementos del árbol y muestra el resultado.

```
// En el main:
arbol.eliminar(20); // Elimina el nodo con valor 20
System.out.println("Inorden después de eliminar 20:");
arbol.inorden(); // Muestra: 30 40 50 60 70 80
```

- Realiza búsquedas en el árbol y muestra el resultado.

```
// En el main:
System.out.println("¿Existe 40? " + arbol.buscar(40)); // true
System.out.println("¿Existe 100? " + arbol.buscar(100)); // false
```

- Implementa y demuestra los recorridos **preorden**, **inorden** y **postorden**.

```
// En el main:
System.out.println("Recorrido Inorden:");
arbol.inorden(); // 20 30 40 50 60 70 80

System.out.println("Recorrido Preorden:");
arbol.preorden(); // 50 30 20 40 70 60 80

System.out.println("Recorrido Postorden:");
arbol.postorden(); // 20 40 30 60 80 70 50
```

Resultados

```
Recorrido Inorden:
20 30 40 50 60 70 80
Recorrido Preorden:
50 30 20 40 70 60 80
Recorrido Postorden:
20 40 30 60 80 70 50
¿Existe 40? true
¿Existe 100? false
Inorden después de eliminar 20:
30 40 50 60 70 80
PS C:\Users\SANTANA\Desktop\Estruct
```

3. Caso práctico: Gestión de empleados

- Proporciona una lista de empleados con sus identificadores únicos (ID).

```
// ESTÁ EN: Clase MainEmpleados - Líneas 8-17
Empleado[] empleados = {
    new Empleado(105, "María López", "Gerente"),
    new Empleado(102, "Carlos Ruiz", "Desarrollador"),
    new Empleado(108, "Ana García", "Diseñadora"),
    new Empleado(101, "Pedro Martínez", "Analista"),
    new Empleado(107, "Laura Díaz", "Recursos Humanos"),
    new Empleado(103, "Juan Pérez", "Contador"),
    new Empleado(106, "Sofía Castro", "Marketing"),
    new Empleado(104, "Miguel Ángel", "Soporte Técnico")
};
// Cada empleado tiene un ID único: 101, 102, 103, etc.
```

- Implementa un sistema donde los empleados se inserten en un árbol binario basado en su ID.

```
public ArbolEmpleados() {
    this.raiz = null;
}

// Insertar empleado
public void insertarEmpleado(Empleado empleado) {
    raiz = insertarRecursivo(raiz, empleado);
}

private NodoEmpleado insertarRecursivo(NodoEmpleado actual, Empleado empleado) {
    if (actual == null) {
        return new NodoEmpleado(empleado);
    }

    if (empleado.id < actual.empleado.id) {
        actual.izquierdo = insertarRecursivo(actual.izquierdo, empleado);
    } else if (empleado.id > actual.empleado.id) {
        actual.derecho = insertarRecursivo(actual.derecho, empleado);
    }

    return actual;
}
```

- Realiza operaciones de búsqueda para encontrar empleados por su ID.

```
// Buscar empleado por ID
public Empleado buscarEmpleado(int id) {
    return buscarRecursivo(raiz, id);
}

private Empleado buscarRecursivo(NodoEmpleado actual, int id) {
    if (actual == null) {
        return null;
    }

    if (id == actual.empleado.id) {
        return actual.empleado;
    }

    if (id < actual.empleado.id) {
        return buscarRecursivo(actual.izquierdo, id);
    } else {
        return buscarRecursivo(actual.derecho, id);
    }
}
```

- Demuestra cómo el sistema mejora la eficiencia en la gestión de empleados en comparación con una búsqueda secuencial.

```
for (int id : idsABuscar) {
    System.out.println("\nBuscando empleado con ID: " + id);

    // Búsqueda en árbol binario
    long inicioArbol = System.nanoTime();
    Empleado resultadoArbol = arbolEmpleados.buscarEmpleado(id);
    long finArbol = System.nanoTime();

    // Búsqueda secuencial
    long inicioSecuencial = System.nanoTime();
    Empleado resultadoSecuencial = BusquedaSecuencial.buscarEmpleadoSecuencial(listaEmpleados, id);
    long finSecuencial = System.nanoTime();

    // Mostrar resultados
    System.out.println("Árbol binario: " + resultadoArbol);
    System.out.println("Tiempo: " + (finArbol - inicioArbol) + " nanosegundos");

    System.out.println("Búsqueda secuencial: " + resultadoSecuencial);
    System.out.println("Tiempo: " + (finSecuencial - inicioSecuencial) + " nanosegundos");
}
```

Resultados

```
=== SISTEMA DE GESTIÓN DE EMPLEADOS ===  
  
Insertando empleados...  
=== LISTA DE EMPLEADOS (Ordenados por ID) ===  
ID: 101 - Pedro Martínez (Analista)  
ID: 102 - Carlos Ruiz (Desarrollador)  
ID: 103 - Juan Pérez (Contador)  
ID: 104 - Miguel Ángel (Soporte Técnico)  
ID: 105 - María López (Gerente)  
ID: 106 - Sofía Castro (Marketing)  
ID: 107 - Laura Díaz (Recursos Humanos)  
ID: 108 - Ana García (Diseñadora)  
  
=== BÚSQUEDAS ===  
  
Buscando empleado con ID: 101  
Árbol binario: ID: 101 - Pedro Martínez (Analista)  
Tiempo: 20600 nanosegundos  
Búsqueda secuencial: ID: 101 - Pedro Martínez (Analista)  
Tiempo: 2156000 nanosegundos  
  
Buscando empleado con ID: 107  
Árbol binario: ID: 107 - Laura Díaz (Recursos Humanos)  
Tiempo: 6400 nanosegundos  
Búsqueda secuencial: ID: 107 - Laura Díaz (Recursos Humanos)  
Tiempo: 12800 nanosegundos  
  
Buscando empleado con ID: 110  
Árbol binario: null  
Tiempo: 6000 nanosegundos  
Búsqueda secuencial: null  
Tiempo: 13500 nanosegundos
```

4. Reporte de la actividad:

- Incluye el código fuente desarrollado.
- Explica el funcionamiento del código.
- Presenta las evidencias de la ejecución del programa (capturas de pantalla o logs).
- Incluye una reflexión sobre la importancia y aplicaciones de los árboles binarios en programación.

Los árboles binarios son estructuras que organizan información de manera eficiente. Su valor principal está en permitir búsquedas, inserciones y eliminaciones de datos mucho más rápidas que en las listas convencionales.

Aplicaciones principales:

- Bases de datos (para buscar registros rápido)
- Diccionarios y sistemas de mapas
- Compresión de archivos
- Procesamiento de operaciones matemáticas
- Sistemas de inteligencia artificial

En esencia, los árboles binarios optimizan el manejo de datos, haciendo que los programas funcionen mejor y más rápido. Son herramientas básicas pero poderosas en programación.

